

Guya — Device-Aware Persian & English Speech-to-Text for Accessibility

B.Sc. Final-Year Project Proposal — Computer Engineering

Student	MohammadReza Ganji
Student ID	220701091
Institution	University of Tehran, Department of Computer Engineering
Supervisor	Dr. Hossein Aghababa, Ph.D. (Electrical & Electronics Engineering)
Date	June 2026
Repository	github.com/faceesmart/guya

1. Overview

Guya (گویا — Persian for "*articulate / speaking*") is a desktop speech-to-text (STT) assistant that lets a user dictate text into any application by holding a key and speaking, in **Persian or English**. It is aimed at people for whom typing long passages on a keyboard is slow, tiring, or difficult.

Its distinguishing idea is that it is **device-aware**: on first run it measures the user's computer and automatically selects the speech model that gives the best accuracy the machine can run responsively — falling back to a **free online model** when the device is too weak. This makes a high-quality assistive tool usable on the low-end hardware that disadvantaged users often have, not only on powerful machines.

A working prototype (offline, online, and a hybrid "switch-on-the-fly" mode) has already been implemented and tested; the remaining work centers on **simple, accessible installation and UX**, and on a **real evaluation with target users**.

2. Problem & Motivation

Keyboard text entry is a daily barrier for many people with disabilities. For users who can speak but find sustained typing effortful, voice dictation can dramatically reduce friction. However, existing solutions fall short for this population in three ways:

1. **Persian is under-served.** Most consumer dictation tools target English; open Persian STT quality is uneven, and informal/colloquial Persian is rarely handled well.
2. **Good models need strong hardware.** The most accurate offline speech models require a GPU or a powerful CPU — hardware many users do not have. Tools rarely adapt to the device.
3. **Setup and interfaces are not accessible.** Installation and configuration often assume technical skill, and interfaces are not designed for users with cognitive disabilities or for a caregiver setting the tool up on their behalf.

This project is also **personally motivated**: the author's own brother is disabled — he can speak and use a keyboard, but writing long text is a constant source of friction. Guya is designed to solve that concrete, real problem.

3. Objectives

1. A reliable push-to-talk **Persian/English dictation** tool that types into any application.
2. A **device-aware model-selection engine** that benchmarks the user's machine and recommends the best feasible model, with a **free online fallback** for weak devices.
3. **Accessible installation and UX** — simple enough to be completed by the user or an assistant in a few clear steps, and calm/low-friction to use daily.
4. An **evaluation** of accuracy, latency, and **real-world usability with disabled users**.

4. Proposed Approach & System Architecture

Guya is built in Python with a Qt interface and the open-source **Whisper** family of models (via *faster-whisper* for local inference and a free cloud API for online use). It has four cooperating parts:

- **Capability profiler & benchmark.** Reads the device (CPU/RAM/GPU) *and* measures its real throughput by timing a small proxy model, then predicts the latency of each candidate model on *this* machine.
- **Model-selection engine.** Chooses the most accurate model that meets a **per-language quality floor** (Persian requires a stronger model than English) **and** a latency budget; otherwise it recommends the **online** model. This yields three modes: **Offline**, **Online (free cloud)**, and **Dual** (both, switchable while in use).
- **Persian post-processing.** A lightweight pipeline (Arabic→Persian character normalization, colloquial-form preservation, a domain correction dictionary, and hallucination filtering) that improves Persian output quality.
- **Setup wizard & floating widget.** A bilingual (English/Persian, right-to-left) guided setup, and a minimal always-on-top widget for push-to-talk dictation.

5. Key Contributions (Novelty)

1. **Benchmark-driven, language-aware model selection.** Rather than guessing from hardware specs, Guya *measures* the device and selects per the user's language need — automatically routing weak devices + Persian to the cloud. This is the core technical contribution.
2. **Offline ↔ online adaptivity** with a user-controlled switch, analyzing the privacy/accuracy/latency trade-off.
3. **Persian (low-resource) handling** through a targeted post-processing pipeline and forced-language decoding.
4. **Accessibility-centered design and evaluation** — including a usability study with disabled users, which is rarely done at this level.

6. Evaluation Plan

- **Accuracy.** Word Error Rate on a Persian (and English) test set, comparing offline tiers vs. the cloud model — quantifying how much accuracy a weak-device user gains by going online.
- **Performance.** Latency / real-time factor across device classes; validation of the benchmark's latency predictions against measured reality.
- **Usability (the key study).** A small study with target users — including the author's brother — measuring task completion time vs. typing, error rates, and a standardized usability score (e.g., SUS), with qualitative

feedback.

7. Scope

In scope (V1): Persian/English dictation; device-aware offline/online/dual selection; simple accessible installation and UX; the evaluation above. Primary platform: **Windows** (cross-platform code; developed and tested on macOS).

Future work: hands-free / wake-word activation; voice-driven editing ("delete that", "new line"); adaptation to atypical/impaired speech; a full control panel.

8. Timeline (≈ 4 weeks remaining)

A working prototype is already complete, so most of the build is done; the remaining effort is on accessible installation/UX and evaluation.

Phase	Status	Deliverable
Proposal & scope approval	✓ Done	This document
V1 core (offline / online / dual)	✓ Done	Working prototype
Accessible installation (one-click installer)	Week 1	No-terminal setup
Accessible UX pass ("Easy mode" + polish)	Week 2	Caregiver-friendly setup & calm UI
Evaluation (accuracy, latency, user study)	Weeks 3–4	Results & analysis
Thesis write-up & demo	Ongoing	Final report + demo video

9. Expected Outcomes

A working, openly-available assistive dictation tool that adapts to the user's hardware and language, is simple enough for disabled users (or a caregiver) to install and use, and is **validated with real target users** — together with a written analysis of the accuracy/latency/privacy trade-offs of device-aware offline-vs-online speech recognition for a low-resource language.

Tools & open-source components: Python, PyQt6, OpenAI Whisper (via faster-whisper), and a free cloud Whisper API. Source code is openly available at the repository above.